

Team 3: New-Hire Discovery Project

Shared Instructions for All Teams

You are being asked to complete a short, realistic project using InterSystems technology before formal product training. The goal is not to create a polished, production-ready application. The goal is to see how you approach a new technology, what assumptions you bring, what resources you discover, and where the new user experience creates friction.

You may use public documentation/learning, Developer Community content, templates, examples, and AI coding assistants. You may ask for help with environment access or credentials, but you should not expect step-by-step implementation guidance.

At the end of the project, your team will be expected to deliver the following in a presentation of your experience that is 10 minutes or less:

- What your team attempted to build
- Where and how the assigned InterSystems technology was used
- The main workflows, outputs, or analyses you created
- What worked well
- What was difficult, confusing, or unexpectedly time-consuming
- What resources, documentation, examples, templates, or AI coding assistants you used
- What you would want fixed, clarified, or provided for future cohorts

After you present your project's results, a group discussion will follow. InterSystems stakeholders will ask about your experience.

During the project, you are encouraged to log your friction points in real time using the friction log for your team: [XDP Friction Log - Team 3.xlsx](#)

Project B: NYC Taxi Trip Insights (Python)

Scenario

New York City publishes detailed trip records for green taxi rides, including pickup and drop-off times and locations, fares, tips, trip distances, payment types, and other trip information.

Transportation analysts want a lightweight application that helps them understand taxi activity while also identifying records that may contain questionable or unusual data.

Your team will assume the role of **Python application developers**. You have been asked to build a prototype using Python and InterSystems IRIS that loads, validates, enriches, and analyzes a full year of NYC green taxi trip data.

Data Set

Use the supplied NYC-Green-Taxi-2023.csv data set in [your project directory](#).

The file contains approximately 787,000 green taxi trips from 2023 and includes fields such as pickup and drop-off timestamps, pickup and drop-off location IDs, passenger count, trip distance, fare amount, tip amount, payment type, and total amount.

Also use the supplied **taxi_zone_lookup.csv** file, which maps taxi location IDs to meaningful borough and zone names.

The trip data has not been pre-cleaned. Part of your task is to investigate the data and decide how questionable or unusual records should be identified and handled.

Your Goal

Your application should:

- Load the supplied taxi data into an InterSystems IRIS Community Edition instance that you set up.
- Use **Python as the primary implementation language** for meaningful parts of the prototype, such as data loading, validation, enrichment, analysis, or application logic.
- Use InterSystems IRIS as the application's data platform.
- Use the Taxi Zone Lookup data to enrich pickup or drop-off locations with meaningful borough or zone information.
- Create a **trip-quality workflow** that identifies records that may deserve further investigation. For example, you might look for:

- Invalid or unusual trip durations
- Zero or negative trip distances
- Negative or unusually high fares
- Unusual relationships between trip distance, time, and fare
- Provide at least **two additional useful user workflows**, such as:
 - Find the busiest pickup or drop-off zones
 - Summarize taxi activity by hour, day, or month
 - Compare fares, tips, or trip distances across zones
 - Investigate common origin/destination pairs
- Present results through an interface of your choice; e.g., a Python notebook, script, command-line tool, dashboard, lightweight UI, or REST API.
- Demonstrate how Python and IRIS together make the data more useful than working directly with the raw files.

Optional Stretch Goal:

Choose one analytical workflow; experiment with performing more of the filtering, aggregation, or analysis in **InterSystems IRIS** rather than retrieving all data into Python.

Compare the two approaches and consider questions such as:

- How much data needs to move between IRIS and Python?
- Which approach is easier to develop and maintain? Why?
- Which approach would you prefer if the dataset became significantly larger?

The goal of the stretch exercise is to explore how Python applications can take advantage of database-side processing rather than treating IRIS only as a place to store data.

Environment & Tooling Recommendations

For Project B (Python), the recommended environment/stack is:

- InterSystems IRIS Community Edition
 - Download from evaluation.intersystems.com
 - If you know Docker, you can also run a Community Edition container pulled from the [InterSystems Container Registry](https://intersystemscontainers.github.io/)
- Visual Studio Code
- A supported Python runtime
- Other tooling as you see fit

You should all have access to Claude Code for this project; the AI enablement team should have sent you your key via Teams. Reach out to Derek Robinson if you do not have yours.

If you have any questions about the environment, tooling, or other access issues, please reach out to Derek Robinson (derek.robinson@intersystems.com, or message Derek on Teams).